✅ MD5 (Message Digest Algorithm 5)

Definition:

MD5 (Message Digest 5) is a **cryptographic hash function** that converts any input message into a **fixed-length 128-bit hash value**, mainly used to ensure **data integrity**.

It was developed by Ron Rivest in 1991.

Basic Concept:

- Input: Any size data (text, file, etc.)
- Output: Fixed **128-bit hash (message digest)**
- Even a small change in input → completely different output

Working of MD5 (Step-by-Step):

1. Padding

- Message is extended so its length becomes **448 mod 512**
- Extra bits are added (1 followed by 0s)

---

2. Append Length

- Original message length (in bits) is added at the end
- Final size becomes multiple of **512 bits**

---

3. Initialize Buffers

- Four 32-bit variables are initialized:
  - A
  - B
  - C
  - D

---

4. Process Message in Blocks

- Message is divided into **512-bit blocks**
- Each block is processed in **4 rounds (16 operations each = 64 operations)**

CONFIDENTIAL

- Uses logical functions like:
    - AND, OR, XOR, NOT

---

5. Output Generation

- Final values of A, B, C, D are combined
- Produces a **128-bit hash value**

Diagram (Important for Exam):

Input Message → Padding → Append Length → Initialize Buffers → Process Blocks → 128-bit Hash Output

---

Example:

- Input: hello
- Output: 5d41402abc4b2a76b9719d911017c592

👉 If input changes to Hello → completely different hash

Properties of MD5:

- Fixed output length (128-bit)
- Deterministic (same input → same output)
- One-way function
- Fast computation

Advantages:

- Simple and fast
- Easy to implement
- Used for **file integrity checking**

Disadvantages:

- Vulnerable to **collision attacks**
- Not secure for modern cryptography
- Can be broken by attackers

CONFIDENTIAL

Applications:

- File verification (downloads)
- Checksums
- Digital fingerprinting (older systems)

Conclusion:

MD5 is a fast hashing algorithm useful for **data integrity**, but due to security weaknesses, it is **no longer recommended for cryptographic security purposes**.

✅ MAC (Message Authentication Code)

Definition:

MAC (Message Authentication Code) is a **cryptographic technique** used to ensure **data integrity and authentication** by generating a fixed-size code using a **secret key and message**.

Basic Idea (Simple Understanding 😉):

👉 Sender and receiver share a **secret key**
👉 Sender creates a **MAC value (tag)** from message
👉 Receiver checks MAC to verify:

- Message is not changed
- Message is from valid sender

Working of MAC (Step-by-Step):

1. At Sender Side

1. Take message (M)
2. Apply MAC algorithm with **secret key (K)**
3. Generate MAC value (Tag)
4. Send:
    - Message + MAC

---

2. At Receiver Side

1. Receive message + MAC

CONFIDENTIAL

2. Apply same MAC function using same key
3. Generate new MAC
4. Compare both MAC values
5. If same → **Valid message**, else **Rejected**

Diagram (Important for Exam):

Sender:

Message + Secret Key → MAC Algorithm → MAC → Send (Message + MAC)

Receiver:

Message + Secret Key → MAC Algorithm → MAC → Compare → Accept / Reject

---

Formula:

MAC = F(K, M)

Where:

- K = Secret Key
- M = Message

Properties:

- Provides **Integrity** (data not modified)
- Provides **Authentication** (sender verified)
- Uses **secret key**
- Fixed-size output

Advantages:

- Simple and efficient
- Protects against data tampering
- Faster than digital signatures

Disadvantages:

- Does not provide **non-repudiation**

CONFIDENTIAL

- Requires secure key sharing
- If key is leaked → system compromised

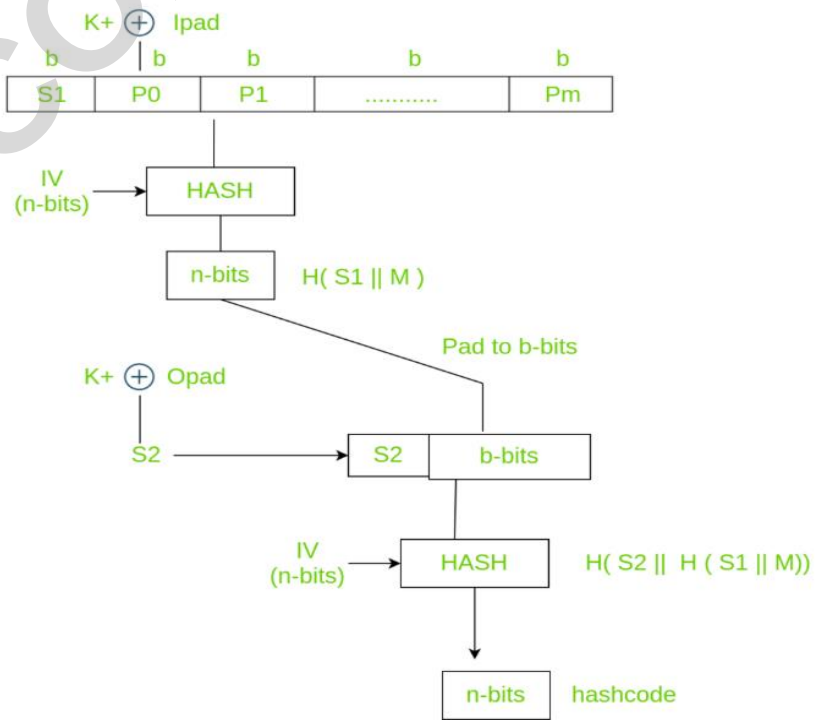
Applications:

- Secure communication
- Banking systems
- Network protocols
- Data transmission

Conclusion:

MAC is an important security mechanism used to ensure **data integrity and authentication**, but it requires **secure key management** and does not provide non-repudiation.

✅ HMAC (Hash-based Message Authentication Code)



Definition:

HMAC (Hash-based Message Authentication Code) is a cryptographic technique that uses a **secret key and a hash function** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

CONFIDENTIAL

👉 Normal MAC = message + key
👉 HMAC = **hash function + secret key + message**

It improves security by using **two layers of hashing**.

HMAC Formula (Important):

🔷 HMAC Formula

$$HMAC(K, M) = H\big((K \oplus opad) \,||\, H((K \oplus ipad) \,||\, M)\big)$$

Where:

- K = Secret key
- M = Message
- H = Hash function (like MD5/SHA)
- opad = Outer padding
- ipad = Inner padding
- ⊕ = XOR operation
- || = Concatenation

🔄 **Full Process of HMAC (Step-by-Step)**

1. Key Preparation

- If key is long → shorten it
- If short → pad with zeros

---

2. Create Inner Key

- K ⊕ ipad (inner padding)

---

3. Inner Hashing

- Concatenate with message:
  👉 (K ⊕ ipad) || Message
- Apply hash function → **Inner Hash**

---

4. Create Outer Key

CONFIDENTIAL

- K ⊕ opad (outer padding)

---

5. Outer Hashing

- Concatenate:
  ☞ (K ⊕ opad) || Inner Hash
- Apply hash function again

---

6. Final Output

- Result = **HMAC value (MAC tag)**

Diagram (Very Important for Exam):

Step 1: K ⊕ ipad → + Message → Hash → Inner Hash

Step 2: K ⊕ opad → + Inner Hash → Hash → Final HMAC

Properties:

- Provides **Integrity**
- Provides **Authentication**
- Resistant to attacks
- Uses secret key

Advantages:

- More secure than simple MAC
- Protects against collision attacks
- Widely used in real systems

Disadvantages:

- Requires secure key sharing
- Slightly slower than simple MAC

Applications:

- HTTPS (SSL/TLS)

CONFIDENTIAL

- API authentication
- Secure communication

Conclusion:

HMAC is a secure and efficient method for ensuring **message integrity and authentication**, widely used in modern security protocols.

✅ CMAC (Cipher-based Message Authentication Code)

Definition:

CMAC (Cipher-based Message Authentication Code) is a cryptographic technique that uses a **block cipher (like AES)** along with a **secret key** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

👉 Instead of using a hash function (like HMAC), CMAC uses **encryption algorithm (AES)**
👉 It generates a **MAC value (tag)** to verify the message

🔄 Working of CMAC (Step-by-Step)

1. Key Generation

- A secret key **K** is shared between sender and receiver

---

2. Subkey Generation

- Using AES, two subkeys are generated:
  - **K1**
  - **K2**

---

3. Message Preparation

- Message is divided into **fixed-size blocks (128-bit)**
- If last block is incomplete → padded

---

4. Processing Blocks

- Each block is processed using **XOR and AES encryption**

CONFIDENTIAL

- Intermediate result is updated after each block

---

5. Final Block Processing

- If last block is complete → XOR with **K1**
- If incomplete → XOR with **K2**

---

6. Final Output

- Final encrypted result = **CMAC (MAC tag)**

📊 **Diagram (Important for Exam):**

Message → Divide into Blocks → XOR → AES Encrypt → Repeat for all blocks

↓

Final Block + K1/K2

↓

AES Encrypt

↓

CMAC Output

🔑 **Properties:**

- Provides **Integrity**
- Provides **Authentication**
- Uses **block cipher (AES)**
- Fixed-length output

✅ **Advantages:**

- More secure than simple MAC
- Based on strong encryption (AES)
- Suitable for hardware implementation
- Standardized and widely accepted

❌ **Disadvantages:**

CONFIDENTIAL

- Slower than simple MAC
- Requires key management
- Depends on cipher strength

📌 **Applications:**

- Secure communication systems
- Banking and financial systems
- Wireless security (e.g., LTE)
- Data authentication

Conclusion:

CMAC is a secure and efficient authentication method that uses **block cipher encryption** to ensure **data integrity and authentication**, making it suitable for modern secure systems.

CONFIDENTIAL